ANÁLISIS DEL RETO

Gabriel Andrés Moya Caravaca-202616964- g.moyac@uniandes.edu.co

Edgar Daniel Gonzalez Martinez-202524741- ed.gonzalezm1@uniandes.edu.co

Gerardo Rocha Linares-202513388- g.rochal2@uniandes.edu.co

Requerimiento 1

Descripción

El requerimiento fue implementado por Gabriel Andrés Moya Caravaca.

Para este requerimiento se utiliza una structura previamente organizada en el catálogo, donde los computadores están agrupados por marca y, dentro de cada marca, por factor de forma. Esto permite acceder directamente a los equipos que cumplen la condición sin recorrer toda la lista. Se obtiene la lista correspondiente a la marca y factor de forma solicitados. Luego se crea una copia de esta lista para evitar modificar la original. Sobre esta copia se calcula el total de computadores y el promedio de precios. Finalmente, la lista se ordena de manera descendente por precio y, en caso de empate, por modelo en orden ascendente. Cabe resaltar, que el algoritmo de ordenamiento empleado fue merge sort.

Entrada	Marca del computador y factor de forma.
Salidas	Número total de computadores, precio promedio y lista de computadores ordenada.
Implementado (Sí/No)	Si, fue implementado por Gabriel Andrés Moya Caravaca.

Análisis de complejidad

La solución es rápida porque no tiene que recorrer todos los computadores, ya que usa una tabla de hash para ir directo a los que cumplen con la marca y el factor de forma. Después de eso, lo que se hace es copiar la lista y calcular el promedio, lo cual depende de cuántos computadores haya en ese grupo. Sin embargo, lo que más tiempo toma es ordenar la lista, ya que se usa merge sort, que tiene una complejidad de O(nlogn). Sin embargo merge trabaja sobre la lista de filtrados lo que transformaría la complejidad en O(klogK) siendo K el número de elementos filtrados. Por otro lado, el cálculo de los promedios solicitados tiene una complejidad de O(k) y la creación de la nueva lista de filtrados también tendría una complejidad de O(K).

Análisis

La solución es eficiente porque accede directamente a los datos mediante la tabla de hash, evitando recorrer todo el catálogo. El mayor costo está en el ordenamiento, pero solo se aplica sobre los elementos filtrados.

Requerimiento 2

Descripción

El requerimiento fue implementado por Gerardo Rocha Linares.

Para este requerimiento se aprovecha una estructura que ya agrupa los computadores por número de núcleos y año de lanzamiento. Gracias a eso, no hace falta recorrer todo el catálogo, sino que se puede acceder directamente a la lista que corresponde a la combinación solicitada. Si esa combinación no existe, el requerimiento termina de inmediato. Cuando la lista sí existe, se recorre para copiar sus elementos a una nueva lista, contar cuántos computadores cumplen el filtro y calcular el promedio de su peso. Después, esa copia se ordena de manera ascendente por peso y, en caso de empate, por modelo. Finalmente, si hay más de 20 resultados, se toman los primeros 10 y los últimos 10 y se juntan en una sola lista para retornarlos.

Entrada	Número de núcleos y año de lanzamiento.
Salidas	Número total de computadores que cumplen el filtro, promedio de peso y lista de resultados ordenada. También se retorna el tiempo de ejecución.
Implementado (Sí/No)	Si, fue implementado por Gerardo Rocha Linares.

Análisis de complejidad

Primero se hace la búsqueda del número de núcleos y del año en la estructura creada previamente, esas consultas en el mapa cuestan O(1) en promedio. Luego se recorre la lista encontrada para copiar los computadores y calcular el promedio del peso, lo cual cuesta O(k), donde k es la cantidad de computadores que cumplen el filtro. Después se ordena la copia de la lista con merge sort, y ese es el paso más costoso, con complejidad O(k log k). Finalmente, si hay más de 20 elementos, se crean las sublistas de los primeros y últimos 10 y se copian a una nueva lista, lo cual cuesta O(1) porque siempre se manejan a lo mucho 20 elementos. Por eso, la complejidad final del requerimiento es O(k log k).

Análisis

El requerimiento es bastante eficiente ya que no necesita revisar todos los computadores del catálogo. Como ya existe una estructura organizada por número de núcleos y año de lanzamiento, la búsqueda va directo al grupo de computadores que cumple con esos dos valores. Eso hace que el filtro principal sea rápido y evita trabajo innecesario.

Requerimiento 3

Descripción

El requerimiento fue implementado por Edgar Daniel Gonzalez Martinez

Para este requerimiento se usa una estructura que ya agrupa los computadores según el modelo de GPU y la marca del equipo. Gracias a eso, no hace falta recorrer todo el catálogo, sino que se puede ir directo a la lista que cumple con esas dos condiciones. Después de obtener esa lista, se calcula cuántos computadores cumplen el filtro y el promedio de memoria RAM. Luego, los resultados se ordenan de manera descendente por precio y, si dos equipos tienen el mismo precio, se usa el peso también de manera descendente como criterio de desempate. Finalmente, se toman solo los primeros N registros.

Entrada	El número N de registros a listar, el modelo de GPU y la marca del equipo.
Salidas	Tiempo de ejecución, número total de computadores que cumplen el filtro, promedio de memoria RAM y la lista con el top N de computadores ordenados.
Implementado (Sí/No)	Si, fue implementado por Edgar Daniel Gonzalez Martinez.

Análisis de complejidad

Primero se accede al mapa con el modelo de GPU y luego con la marca, lo cual en promedio cuesta O(1). Después se recorre la lista encontrada para copiar los computadores y calcular el promedio de RAM, con costo O(k). Luego se ordena esa lista, que es el paso más costoso y tiene complejidad O(k log k). Finalmente, se toma la sublista con los primeros N elementos. Por eso, la complejidad total queda dominada por O(k log k), donde k es la cantidad de computadores que cumplen el filtro.

Análisis

La solución es eficiente porque accede directamente a los datos mediante la tabla de hash, evitando recorrer todo el catálogo. El mayor costo está en el ordenamiento, pero solo se aplica sobre los elementos filtrados.

Requerimiento 4

Descripción

El requerimiento fue implementado por Gabriel Andrés Moya Caravaca

Para este requerimiento se usa un mapa que ya agrupa los computadores por CPU Brand y GPU Model, lo que permite ir directo a la lista que interesa. Si la combinación sí existe, se toma la lista de computadores correspondiente y se hace una copia para no modificar la original. Posteriormente, se calculan los promedios (precio, VRAM, RAM y boost). Después, la lista copiada se ordena por precio y, en caso de empate, se usa el peso en orden ascendente como criterio de desempate. Una vez la lista esta ordenada se toman los dos computadores más costosos. Finalmente, se retorna la información calculada junto con los dos computadores mas costosos.

Entrada	Marca de la CPU (CPU brand) y modelo de la GPU (GPU model)
Salidas	Número de computadores que cumplen, precio promedio, VRAM promedio, RAM promedio, promedio de velocidad boost y los dos computadores más costosos.
Implementado (Sí/No)	Si, fue implementado por Gabriel Andrés Moya Caravaca.

Análisis de complejidad

Primero se accede al mapa usando la combinación de CPU Brand y GPU Model, lo cual en promedio toma tiempo constante O(1). Luego se copia la lista de computadores encontrados, lo que implica recorrer todos los elementos y tiene un costo de O(k). Después, se vuelve a recorrer esa misma lista para calcular los promedios de precio, VRAM, RAM y velocidad boost, lo cual también cuesta O(k). Finalmente, se ordena la lista usando merge sort, que es el paso más costoso con una complejidad de O(k log k), donde k es la cantidad de computadores que cumplen con la combinación solicitada.

Análisis

La solución funciona bien porque usa la tabla de hash para agrupar los computadores y evitar búsquedas innecesarias. Eso hace que encontrar los equipos que cumplen con el número de núcleos y el año sea mucho más directo. El paso más costoso termina siendo el ordenamiento, pero solo se aplica sobre los computadores filtrados y no sobre todo el catálogo, así que sigue siendo una solución razonable.

Requerimiento 5

Descripción

El requerimiento fue implementado por Gerardo Rocha Linares.

Para este requerimiento se aprovechó una estructura que ya agrupa los computadores por marca y factor de forma. La idea fue usar esa organización para no tener que revisar todo el catálogo, sino ir directo al grupo de computadores que coincide con los valores que se piden. Una vez obtenida esa lista, se filtraron solo los equipos que estuvieran dentro del rango de años solicitado. mientras se hacía ese filtrado, también se contaron los computadores que cumplían el filtro y se separó cuántos tenían procesador intel y cuántos AMD. Después, los equipos filtrados se ordenaron según lo pedido: primero por memoria RAM de mayor a menor, luego por velocidad de CPU también de mayor a menor, y en caso de empate, por precio de menor a mayor. Al final, solo se tomaron los primeros N computadores de la lista ya ordenada.

Entrada	El número N de registros a listar, año de lanzamiento inicial, año de lanzamiento final, marca (brand) y factor de forma (form_factor).
Salidas	Cantidad de computadores con procesador AMD, cantidad de computadores con procesador Intel, lista con los N computadores mejor equipados, total de computadores que cumplen el filtro y tiempo de ejecución.
Implementado (Sí/No)	Si, fue implementado por Gerardo Rocha Linares

Análisis de complejidad

Primero se accede al mapa de la marca y luego a la lista asociada al factor de forma, lo cual en promedio cuesta O(1). Después se recorre esa lista para filtrar por rango de años y contar los computadores Intel y AMD, con costo O(k), donde k es la cantidad de computadores que pertenecen a esa marca y factor de forma. Luego se ordena la lista filtrada, y ese es el paso más costoso, con una complejidad de O(m log m), donde m es la cantidad de computadores que sí cumplen el rango de años. Finalmente, tomar los primeros N elementos cuesta O(N). Por eso, la complejidad final seria O(k + m log m).

Requerimiento 6

Descripción

El requerimiento fue implementado por Edgar Daniel Gonzalez Martinez

Para este requerimiento se usa una estructura que agrupa los computadores por factor de forma y tipo de pantalla, lo que permite acceder directamente a los equipos que cumplen esas condiciones. A partir de esa lista, se filtran únicamente los computadores portátiles que estén dentro del rango de años solicitado. Sobre cada computador filtrado se calcula un puntaje de eficiencia usando la relación entre la batería, la velocidad del CPU y el cargador. Luego se cuenta cuántos equipos usan Windows y cuántos Linux. Finalmente, los computadores se ordenan de mayor a menor según este puntaje y, en caso de empate, se usa el precio como criterio de desempate en orden ascendente. Después se toman los primeros N resultados.

Entrada	Número N de computadores, factor de forma, tipo de pantalla y rango de años.
Salidas	Tiempo de ejecución, número total de computadores que cumplen el filtro, cantidad de equipos con Windows, cantidad con Linux y el top N de computadores ordenados por eficiencia.
Implementado (Sí/No)	Si, fue implementado por Edgar Daniel Gonzalez Martinez.

Análisis de complejidad

Primero se accede al mapa usando el factor de forma y tipo de pantalla, lo cual en promedio cuesta O(1). Luego se recorre la lista para filtrar los computadores, calcular el puntaje de eficiencia y contar los sistemas operativos, con costo O(k). Después se debe ordenar la lista según el puntaje de eficiencia, lo cual cuesta O(k log k). Por eso, la complejidad total queda dominada por O(k log k), donde k es la cantidad de computadores que cumplen el filtro.

REQUERIMIENTOS:

```python
def req_1(catalog, brand, form_factor):
    """
    Retorna el resultado del requerimiento 1
    """
    brand = brand.strip().upper()
    form_factor = form_factor.strip().upper()
    inner_map = lp.get(catalog["computers_by_brand_form"], brand)
    if inner_map is None:
        return (0, 0.0, al.new_list())
    lst = lp.get(inner_map, form_factor)
    if lst is None:
        return (0, 0.0, al.new_list())

    copied_list = al.new_list()
    for i in range(al.size(lst)):
        al.add_last(copied_list, al.get_element(lst, i))

    total = al.size(copied_list)

    sum_price = 0.0
    count = 0
    for i in range(total):
        comp = al.get_element(copied_list, i)
        price = 0.0
        if comp.get("price"):
            price = float(comp["price"])
            sum_price += price
            count += 1
    avg_price = sum_price / count if count > 0 else 0.0

    al.merge_sort(copied_list, sort_by_price_desc_model_asc)

    return (total, avg_price, copied_list)
```

REQUERIMIENTO 1

```python
def req_2(catalog, nucleos, año_de_lanzamiento):
    cumplen=0
    peso_total=0
    inicio=getTime()
    mapa=lp.get(catalog["computers_by_cores_year"],nucleos)

    if mapa is None:
        fin=getTime()
        return 0, 0, None, deltaTime(inicio, fin)
    lista=lp.get(mapa,año_de_lanzamiento)
    if lista is None:
        fin=getTime()
        return 0, 0, None, deltaTime(inicio, fin)

    copia=al.new_list()
    tamaño=al.size(lista)

    for i in range(tamaño):
        computador=al.get_element(lista,i)
        al.add_last(copia,computador)
        peso=float(computador["weight_kg"])
        peso_total+=peso
        cumplen+=1

    peso_promedio=peso_total/cumplen

    al.merge_sort(copia,sort_by_weight)
    fin=getTime()

    if al.size(copia)>20:
        primeros_10=al.sub_list(copia,0,10)
        ultimos_10=al.sub_list(copia,cumplen-10,10)
        primeros_y_ultimos=al.new_list()
        for i in range(al.size(primeros_10)):
            comp = al.get_element(primeros_10,i)
            al.add_last(primeros_y_ultimos, comp)

        for i in range(al.size(ultimos_10)):
            comp = al.get_element(ultimos_10,i)
            al.add_last(primeros_y_ultimos, comp)
        return cumplen, peso_promedio, primeros_y_ultimos, deltaTime(inicio, fin)

    return cumplen, peso_promedio, lista, deltaTime(inicio, fin)
```

REQUERIMIENTO 2

```python
def req_3(catalog, n, gpu_model, brand):

    """
    Retorna el resultado del requerimiento 3
    """
    # TODO: Modificar el requerimiento 3
    gpu_model = gpu_model.strip().upper()
    brand = brand.strip().upper()
    Inicio = getTime()



    GPU = lp.get(catalog["computers_by_gpu_brand"], gpu_model)

    lista_filtrada = al.new_list()
    ram = 0

    if GPU is not None:
        gpu_list = lp.get(GPU, brand)

        if gpu_list is not None:
            for i in range(al.size(gpu_list)):
                comp = al.get_element(gpu_list, i)
                al.add_last(lista_filtrada, comp)
                ram += int(comp.get("ram_gb", 0))



    total = al.size(lista_filtrada)
    if total > 0:
        promedio_ram = ram / total
    else:
        promedio_ram = 0

    def criterio_ordenamiento(comp1, comp2):
        precio1 = float(comp1.get("price"))
        precio2 = float(comp2.get("price"))

        if precio1 > precio2:
            return True
        elif precio1 < precio2:
            return False
        else:
            peso1 = float(comp1.get("weight_kg"))
            peso2 = float(comp2.get("weight_kg"))
            return peso1 > peso2

    al.merge_sort(lista_filtrada,criterio_ordenamiento)


    if n > total:
        n = total

    resultado_n  = al.sub_list(lista_filtrada, 0, n)

    Final  = getTime()


    Tiempo_final = Final - Inicio

    return {"tiempo": Tiempo_final,"total": total,"promedio_ram": promedio_ram,"resultado_n": resultado_n
    }
```

REQUERIMIENTO 2

```python
def req_4(catalog, cpu_brand, gpu_model):
    """
    Retorna el resultado del requerimiento 4
    """
    cpu_brand = cpu_brand.strip().upper()
    gpu_model = gpu_model.strip().upper()

    inner_map = lp.get(catalog["computers_by_cpu_brand_gpu_model"], cpu_brand)
    # print("\n|-----------------------------|")
    # print(f"Inner map for CPU brand '{cpu_brand}': {inner_map}")
    # print(inner_map)
    # print("|-----------------------------|\n")
    if inner_map is None:
        return (0, 0.0, 0.0, 0.0, 0.0, al.new_list())
    lst = lp.get(inner_map, gpu_model)
    if lst is None:
        return (0, 0.0, 0.0, 0.0, 0.0, al.new_list())


    copied_list = al.new_list()
    for i in range(al.size(lst)):
        al.add_last(copied_list, al.get_element(lst, i))

    total = al.size(copied_list)

    sum_price = 0.0
    sum_vram = 0.0
    sum_ram = 0.0
    sum_boost = 0.0
    for i in range(total):
        comp = al.get_element(copied_list, i)
        price = float(comp["price"]) if comp["price"] else 0.0
        vram = float(comp["vram_gb"]) if comp["vram_gb"] else 0.0
        ram = float(comp["ram_gb"]) if comp["ram_gb"] else 0.0
        boost = float(comp["cpu_boost_ghz"]) if comp["cpu_boost_ghz"] else 0.0
        sum_price += price
        sum_vram += vram
        sum_ram += ram
        sum_boost += boost

    avg_price = sum_price / total if total > 0 else 0.0
    avg_vram = sum_vram / total if total > 0 else 0.0
    avg_ram = sum_ram / total if total > 0 else 0.0
    avg_boost = sum_boost / total if total > 0 else 0.0


    al.merge_sort(copied_list, sort_by_price_desc_weight_asc)

    return (total, avg_price, avg_vram, avg_ram, avg_boost, copied_list)
```

REQUERIMIENTO 4

```python
def req_5(catalog, n, initial_release_year, final_release_year, brand, form_factor):
    inicio=getTime()

    numero_AMD=0
    numero_INTEL=0
    cumplen=0
    tamaño=al.size(catalog["computers"])
    lista_cumplen=al.new_list()

    brand = brand.strip().upper()
    form_factor = form_factor.strip().upper() #el form factor que me intereza
    initial_release_year = int(initial_release_year) #el año en el que inicia el rango que me interesa
    final_release_year = int(final_release_year) #el año en el que finaliza el rango que me interesa

    mapa=lp.get(catalog["computers_by_brand_form"],brand) #saco el mapa de los computadores con la marca y el form factor
    if mapa is None:
        fin=getTime()
        return 0, 0, None, 0, deltaTime(inicio, fin)
    lista=lp.get(mapa,form_factor)
    if lista is None:
        fin=getTime()
        return 0, 0, None, 0, deltaTime(inicio, fin)

    for i in range(al.size(lista)):
        computador=al.get_element(lista,i)
        anio=int(computador["release_year"])
        if initial_release_year <= anio <= final_release_year:
            cumplen+=1
            al.add_last(lista_cumplen,computador)
            if computador["cpu_brand"].strip().upper()=="INTEL":
                numero_INTEL+=1

            elif computador["cpu_brand"].strip().upper()=="AMD":  # reviso que los filtrados sean AMD o INTEL
                numero_AMD+=1

    if cumplen == 0:
        fin=getTime()
        return 0, 0, None, 0, deltaTime(inicio, fin)

    al.merge_sort(lista_cumplen,sort_crit_req_6)

    if n > al.size(lista_cumplen): # si el n solicitado excede el numero de los que cumplen retorno los que cumplieron
        n=al.size(lista_cumplen)

    n_elements=al.sub_list(lista_cumplen,0,n)

    fin=getTime()
    return numero_AMD, numero_INTEL, n_elements, cumplen, deltaTime(inicio, fin)
```

REQUERIMIENTO 5

```python
def req_6(catalog, n, form_factor, display_type, año_inicial, año_final):


    # TODO: Modificar el requerimiento 6
    form_factor = form_factor.strip().upper()
    display_type = display_type.strip().upper()
    Inicio = getTime()

    lista_filtrada = al.new_list()
    contador_windows = 0
    contador_linux = 0
    contador_total = 0

    mapa_pantallas = lp.get(catalog["computers_by_form_display"], form_factor)

    if mapa_pantallas is not None:
        computadoras_pantalla = lp.get(mapa_pantallas, display_type)

        if computadoras_pantalla is not None:
            for i in range(al.size(computadoras_pantalla)):
                comp = al.get_element(computadoras_pantalla, i)

                laptops = comp.get("device_type").strip().upper()
                año = int(comp.get("release_year"))

                if laptops == "LAPTOP" and int(año_final) >= año >= int(año_inicial):
                    battery_wh = float(comp.get("battery_wh"))
                    cpu_boost = float(comp.get("cpu_boost_ghz"))
                    charger_watts = float(comp.get("charger_watts"))

                    puntaje_eficiente = (battery_wh * cpu_boost) / charger_watts
                    comp["efficiency_score"] = puntaje_eficiente
                    al.add_last(lista_filtrada, comp)

                    sistema_operativo = comp.get("os", "").strip().upper()
                    if sistema_operativo == "WINDOWS":
                        contador_windows += 1
                    elif sistema_operativo == "LINUX":
                        contador_linux += 1
    total = al.size(lista_filtrada)

    def criterio_ordenamiento(comp1, comp2):
        puntaje1 = comp1.get("efficiency_score", 0)
        puntaje2 = comp2.get("efficiency_score", 0)

        if puntaje1 > puntaje2:
            return True
        elif puntaje1 < puntaje2:
            return False
        else:
            precio1 = float(comp1.get("price"))
            precio2 = float(comp2.get("price"))
            return precio1 < precio2

    al.merge_sort(lista_filtrada, criterio_ordenamiento)
    resultado_n = al.new_list()
    if n > total:
        n = total

    for i in range(n):
        al.add_last(resultado_n, al.get_element(lista_filtrada, i))
    final = getTime()
    tiempo_final = final - Inicio

    return {"tiempo": tiempo_final, "total": total, "windows": contador_windows, "linux": contador_linux, "resultado_n": resultado_n}
```

REQUERIMIENTO 6